

P-FUNDS

Professional Project Funding

& Milestone Verification Platform

A detailed technical architecture manual, directory mapping, database schema, and mock examiner Q&A designed for project defense preparation.

Created For: Project Defense Panel

Target Roles: Creator, Sponsor, Vetter, and Administrator

Backend: PHP / Laravel 11 | Frontend: Vanilla JS / HTML5 / CSS3

Prepared for: Project Defense Presentation

1. Project Overview & Scope

What is P-FUNDS?

P-FUNDS is a web-based, multi-role platform designed to bridge the gap between **Creators** (Entrepreneurs/Innovators) seeking funding, and **Sponsors** (Investors/Funders) looking for promising projects. Unlike traditional crowdfunding platforms (e.g., Kickstarter) where creators receive all funds upfront with little accountability, P-FUNDS introduces **Vetters** (Review Authorities). Vetters review project details and milestone deliverables before release of funds, ensuring transparency and investor protection.

The Problem it Solves

- 1. Lack of Accountability:** Traditional platforms do not track what creators do with funds after receiving them. P-FUNDS uses **milestones** and **deliverables** to release funds incrementally.
- 2. Investor Risk:** Sponsors often lose money to fraudulent or poorly executed projects. Vetters act as escrow-like auditors to verify milestones.
- 3. Communication Gap:** A global chat and private chat system allow immediate, role-restricted discussions between platform users.

User Roles & Permissions

- 1. Creators:** Can register, create draft projects, upload business/technical documents, submit projects for vetting, and upload deliverables (proof of work) for milestones.
- 2. Sponsors:** Can browse approved projects, save/like/dislike projects, write direct private chats to creators, and commit funds to projects.
- 3. Vetters (Level 1, 2, 3):** Can view the vetting queue, review submitted documents, approve/reject individual milestones, and require project updates.
- 4. Admins:** Oversee the entire system, manage users (suspend/role update), review activity logs, and approve final vetting overrides.

2. Technology Stack

The platform is designed as a **unified monolith with separated layers**. The frontend client's HTML, CSS, and JS files are housed directly within the Laravel project's `public/`` directory. This allows the entire application (both the API endpoints and the dashboard screens) to be served from a single origin/port, eliminating all CORS (Cross-Origin Resource Sharing) restrictions while keeping the interface cleanly decoupled from the backend logic.

Backend: RESTful API



****Framework:**** Laravel 11.x (PHP 8.2+)

- ***Why Laravel?*** It provides robust MVC routing, built-in security features, Eloquent ORM for database queries, database migrations, and clean request validation.
- ****Database:**** MySQL
- ***Why MySQL?*** It is a robust, production-ready relational database management system. It supports high concurrency (multiple vetters, creators, and sponsors writing simultaneously), complex transactions, and structured security controls suitable for financial and milestone releases.
- ****Authentication:**** Laravel Sanctum
- ***Why Sanctum?*** It provides stateless token-based API authentication. When users login, Sanctum generates a unique `Bearer Token` that the client sends in the HTTP headers for subsequent requests.

Frontend: Client Dashboard

- ****Structure & Logic:**** HTML5 & ES6 JavaScript.
- ****Styling:**** Vanilla CSS3. Styling is modularized across `index.css`, `dashboard.css`, and role-specific files (`vetter.css`, `sponsor.css`).
- ****Networking:**** Fetch API. Helper library `api.js` acts as an HTTP client wrapper (defining a global `apiClient`) to make asynchronous HTTP requests.

3. Directory Map & Code Layout

Backend Directory (`P-FUND-APIs/`)

- `routes/api.php` - Defines API routes, applies middleware protection (`auth:sanctum`, `role`).
- `app/Models/` - Classes representing database tables.
- `User.php` - Users and credentials.
- `Project.php` - Project information and status.
- `Milestone.php` - Project steps and approval status.
- `ProjectDeliverable.php` - Documents submitted by creators as proof of milestone completion.
- `Message.php` - Chat log messages.
- `Notification.php` - Topbar alert messages.
- `app/Http/Controllers/Api/V1/` - Request handlers.
- `AuthController.php` - Handles signups, logins, and token issuance.
- `ChatController.php` - Filters and posts chat messages.
- `NotificationController.php` - Handles unread notifications and read actions.
- `VetterProjectController.php` - Manages queue stats and milestone actions for Veters.
- `app/Http/Middleware/EnsureUserHasRole.php` - Intercepts requests to check if the authenticated user's role matches the route restrictions.

Frontend Directory (`P-FUNDS/`)

-

`pages/auth/` - Login, registration, password reset pages.

- `pages/dashboard/` - Portal HTML folders: `admin/`, `sponsor/`, `vetter/`, `user/`.
- `assets/js/` - Shared scripting engines:
- `api.js` - Isolation of local storage, API configuration, and HTTP request routing.
- `chat.js` - Interactive panel widget toggling, message rendering, and background polling.
- `notifications.js` - Loading and clearing unread notifications in topbars.
- `dist/styles/` - Styling assets.

4. Database Schema & Relationships

The database consists of 10 tables structured around the core entities:

1. users:	id, name, email, password, role, friendly_role, is_active, avatar_url, created_at, updated_at
2. projects:	id, user_id, title, description, category, budget_amount, budget_currency, status, created_at, updated_at
3. milestones:	id, project_id, title, description, due_date, status, created_at, updated_at
4. project_documents:	id, project_id, file_path, file_name, file_size, created_at, updated_at
5. project_deliverables:	id, project_id, milestone_id, file_path, file_name, status, feedback, created_at, updated_at
6. messages:	id, user_id, recipient_id, message, created_at, updated_at
7. notifications:	id, user_id, title, body, read_at, created_at, updated_at
8. activity_logs:	id, user_id, action, ip_address, created_at, updated_at

Table Relationships (ORM)

- ****User --> Project (One-to-Many):**** A Creator (`User`) can submit many `Projects`.
- `User::hasMany(Project::class)` | `Project::belongsTo(User::class)`
- ****Project --> Milestone (One-to-Many):**** A `Project` contains multiple `Milestones` that must be achieved.
- `Project::hasMany(Milestone::class)` | `Milestone::belongsTo(Project::class)`
- ****Project --> ProjectDocument (One-to-Many):**** A `Project` contains supporting documents uploaded on creation.
- `Project::hasMany(ProjectDocument::class)`
- ****Project --> ProjectDeliverable (One-to-Many):**** A `Project` requires multiple deliverables to prove milestone achievement.
- `Project::hasMany(ProjectDeliverable::class)`
- ****User --> Message (One-to-Many):**** A `User` can send many `Messages` (chat messages).
- `Message::belongsTo(User::class, 'user\_id')`
- ****User --> Notification (One-to-Many):**** A `User` can receive multiple alert notifications.
-

```
`Notification::belongsTo(User::class)`
```

5. Key System Workflows

A. Authentication & Session Isolation

- 1. Login:** The frontend sends email and password to ``/api/v1/auth/login``.
- 2. Token Generation:** The backend matches credentials, creates a token via Sanctum, and returns it with user details.
- 3. Storage Prefixing:** The client ``api.js`` script intercepts local storage calls. Depending on the path (e.g., ``/dashboard/vetter/``), it saves the keys with a role prefix (``vetter_pfunDS_token``). This prevents session clashing (e.g., logging in as a sponsor in one tab does not corrupt the admin session in another).

B. The Vetting Pipeline

1. Creator builds a project with 3 milestones and submits it (``status = 'vetting'``).
2. Vetter loads the project. The system checks which level of milestone is under review:
 - Level 1 Milestone: Checked and updated by ``vetter_1``.
 - Level 2 Milestone: Checked and updated by ``vetter_2``.
 - Level 3 Milestone: Checked and updated by ``vetter_3``.
3. The Vetter reviews the deliverables, files, and updates the milestone status. If all milestones are completed, the project moves to ``approved`` status, making it visible to sponsors.

C. Chat Visibility & Roles

To prevent conflicts of interest and maintain business boundaries, the public chat has strict role visibility filters built into ``ChatController@index``:

- ****Sponsors**** can only see public messages sent by Admins or other Sponsors (hiding public chatter from creators/vetters).
- ****Vetters & Creators**** can see public messages sent by Admins, other Vetters, or Creators (hiding public chatter from sponsors).
- ****Private Messages**** (where ``recipient_id`` is set) bypass these filters but are strictly limited to the sender and the recipient.

6. Project Defense: Mock Examiner Q&A

Q1: Why did you choose a monolithic structure with separated frontend assets?

Answer: Putting our frontend HTML, CSS, and JS files into Laravel's ``public/`` directory creates a unified, single-port monolithic application. This layout completely eliminates Cross-Origin Resource Sharing (CORS) security issues and allows you to serve both the frontend and the backend API from a single server (via ``php artisan serve``). At the same time, because the frontend is built using standard HTML5/JS files that

consume the `/api/v1` backend endpoints, the presentation layer remains completely decoupled from the PHP business logic, allowing future API reusability.

Q2: What database are you using, and why?

Answer: We are using MySQL. It is an industry-standard, client-server relational database. While SQLite is file-based and locks the entire database on writes, MySQL supports high concurrency, row-level locking, and high-performance querying, which are crucial for a crowdfunding platform where creators, vetters, and sponsors concurrently commit funds, approve milestones, and chat.

Q3: How is user authentication secured?

Answer: Authentication is handled by Laravel Sanctum using stateless tokens. Instead of traditional cookie-based sessions, the server issues a cryptographically secure token string upon login. The client saves this token in local storage and includes it as a header (`Authorization: Bearer <token>`) in every subsequent API request. The server validates this token signature against its database before responding.

Q4: How do you prevent a Creator from modifying someone else's project or approving their own milestones?

Answer: We implement route protection using Laravel **Middleware** and Controller-level authorization checks.

1. At the routing level, middleware restricts specific URL prefixes to specific roles (e.g., `Route::middleware('role:veter')` blocks non-vetters).

2. Inside the controllers, we verify ownership. For example, when updating a project, we check if the authenticated user's ID matches the project's creator ID:

```
`if ($project->user_id !== $request->user()->id) { return response()->json(['message' => 'Unauthorized'], 403); }`
```

Q5: What is CORS and how does this project handle it?

Answer: CORS stands for *Cross-Origin Resource Sharing*. It is a browser security mechanism that blocks web pages from making requests to a different domain than the one that served the page. Since our frontend is served from `http://localhost/P-FUNDS` and the API runs on `http://127.0.0.1:8000`, the browser blocks requests by default. We resolved this by configuring CORS middleware in Laravel to allow requests from the frontend origin, returning the header `Access-Control-Allow-Origin: \*` (or specific client URL) along with allowed headers and methods.

Q6: How does the real-time chat work? Is it using WebSockets?

Answer: The chat system uses **HTTP Polling** rather than WebSockets. While WebSockets (like Pusher or Socket.io) maintain a persistent open connection, they require more system resources and complex setup. Instead, our client script (`chat.js`) runs an asynchronous background polling loop using `setInterval` that sends a GET request to `/api/v1/chat` every 5 seconds when the chat panel is open (and every 15 seconds in the background for notifications). This is simple to implement, extremely reliable, and perfect for prototypes.

Q7: Explain what SQL queries are executed in ChatController for role visibility.

Answer: The query uses sub-query filtering. For public messages (no recipient), it checks the sender's role:

- If the viewer is a sponsor: ``SELECT * FROM messages WHERE recipient_id IS NULL AND EXISTS (SELECT * FROM users WHERE users.id = messages.user_id AND role NOT IN ('vetter', 'creator') AND role NOT LIKE 'vetter_%')``
- If the viewer is a vetter/creator: ``SELECT * FROM messages WHERE recipient_id IS NULL AND EXISTS (SELECT * FROM users WHERE users.id = messages.user_id AND role != 'sponsor')``

This prevents cross-role messaging leakages in public feeds.

Q8: What happens in the database when a Sponsor funds a project?

Answer: When a sponsor calls the ``/projects/{project}/fund`` endpoint:

1. The backend validates that the project is in ``approved`` status.
2. It creates a record in the ``sponsor_commitments`` table containing the ``user_id`` (sponsor), ``project_id``, and ``amount`` funded.
3. It creates an entry in ``sponsor_project_interactions`` logging a ``fund`` interaction.
4. It checks if the total commitments cover the project budget. If yes, it transitions the project status to ``funded``.
5. It triggers a notification to the Creator informing them that funding is secured.

Q9: Why did you override `Storage.prototype` in `api.js`?

Answer: We did this to isolate sessions across different user roles. When testing the platform locally on the same browser, logging in as a Sponsor in one tab and a Vetter in another would overwrite the ``pfunds_token`` in local storage, logging out the previous session. By overriding ``getItem`` and ``setItem`` to append a path-based prefix (e.g., ``vetter_pfund_token``), each dashboard tab gets its own isolated session keys, allowing multi-role testing simultaneously.

Q10: How are passwords secured in the database?

Answer: Passwords are never stored in plain text. Laravel automatically hashes passwords using the **Bcrypt** hashing algorithm when a user registers. Bcrypt is a secure, slow hashing function designed specifically to resist brute-force attacks. During login, Laravel retrieves the hashed password from the database and uses PHP's built-in ``password_verify()`` function to check if the entered password matches the hash.

Q11: What is a Database Migration in Laravel, and why is it useful?

Answer: Database migrations are like version control for the database schema. Instead of sharing SQL files or manually creating tables, we write PHP scripts that define the table structures, columns, and indexes (located in ``database/migrations/``). Running ``php artisan migrate`` executes these scripts, creating the tables consistently across any development machine.

Q12: How are files (like PDF deliverables) stored on the backend?

Answer: Files are uploaded using multipart form requests. The backend saves files securely inside the ``storage/app/public/`` directory (e.g., in a folder named ``documents/`` or ``deliverables/``). The database does

not store the file itself, but rather stores a relative file path string (e.g., ``documents/project_12_proposal.pdf``). To access files, the frontend requests a download endpoint, and the controller streams the file back as a download attachment using Laravel's ``Storage::download()`` helper.

7. Key Defense Presentation Tips

1. ****Understand the Flow:**** Be ready to click through the system in this order: Creator registers --> creates project --> submits --> Vetter logs in --> checks details --> approves milestones --> Sponsor logs in --> funds project --> Creator uploads proof of work --> Vetter approves final milestones.
2. ****Be Clear on Security:**** Emphasize that Sanctum token checks and custom Role Middleware block unauthorized role access at the API level, rendering frontend inspection manipulation useless.
3. ****Know the Code:**** If asked where a specific logic lives, immediately reference the Controller or JS file (e.g., ****"The chat visibility filter logic lives inside the ``ChatController.php`` file on line 24"****).